Beh Zhi Yin **FIT3152-A3** **33561656**

1.

I had collected 21 text files, which have 3 different genres.

The News genre includes articles exclusively focused on Artificial Intelligence (AI), covering topics such as AI's impact on employment, regulatory frameworks, and its use in job interviews.

The Film Review genre centres on films that explore themes of death, and the meaning of life, like *Soul*, *The Farewell*, and *Coco*.

The Political genre is focused on women's participation in politics, including discussions on global trends, challenges, and strategies for improving gender equality in political spaces.

This consistent selection of themes within each genre (especially news and political genres) supports clearer clustering in the rest of the question.

2.

I collected these 21 documents from online and grouped them into three genres - News, Film Review, and Political, with filenames formatted clearly (e.g., news01.txt, filmreview06.txt, political03.txt) for easy genre recognition during analysis. I grouped based on their content reflected. However, one thing to note is that some content in the Political and Film Review genres was sourced from news websites, which might suggest it could be grouped under the News genre. Nevertheless, I ensured that the selected articles were not focused on reporting specific events and people, quoting individuals, or interviewing real people. Instead, they were centred on thematic discussions, such as using study findings to prove it. Therefore, I categorised them under their respective genres, such as Film Review or Political, based on the primary focus of the content.

Each file name follows the format: <genre><numberid>.txt. Such as filmreview01.txt, news03.txt, political06.txt

This consistent naming strategy allows easy tracking of genre and ID, which simplifies further analysis and improves clustering accuracy, and makes sure the title of the document won’t such long when presented as a single node in the graph. I also ensured that each file only corresponds to one genre, which easier to distinguish groupings during clustering.

I also make sure each genre is equally having 7 text files in each particular genre, and I copy paste all the content manually but without also copy the title, authors, images, date of published these kind of information into the word document, and click the save as function and saved them as UTF-8 and .txt file format, which allows me to better analyse these file in the rest of the question in this assignment.

To create my corpus for clustering and topic modelling, I used a folder of plain text files containing the 21 documents from three specific genres: Film Reviews, News, and Political. These texts are also sourced from credible platforms such as *Roger Ebert*, *New York Times*, *The Star*, *UN Women*, and others.

All documents were made sure to be saved in .txt format and placed into a folder named txt, located in the following directory:

C:/Users/zhiyin1018/Desktop/FIT3152/Assignment/A3/CorpusAbstracts/txt/

```
docs <- Corpus(DirSource(cname))
```

I also use Corpus(DirSource(cname)) this line of code to read all .txt files to create my corpus, which is then ready for preprocessing, clustering.

3.

Once the corpus was loaded, I began by eliminating citation markers such as [1], [2], and hyperlinks using regular expressions like www. or https://. Next, I normalised all text to lowercase to ensure that tokens like “Death” and “death” would be treated as the same term. After I had sorted terms which contain the high frequencies, I noticed there are some columns which has repeated meaning terms, such as artifici and intellig. Even I had used the stemwords to do so, but the artificial intelligence with two letters is confusing the DTM. So, I do some keyword transformations to preserve meaning first: variations of “artificial intelligence” (e.g., AI, A.I., a i) were standardised to a custom token “artifici”, while verbs like “say” were converted to their past form “said” to unify verb tense. I also mapped death-related verbs like “died”, “passed away”, and “dying” to the unified keyword “death” to preserve thematic consistency.

Further transformations included removing punctuation, numbers, and a set of customised stopwords that extended the default English stopword list (e.g., “nai”, “one”, “also”, “two”), as these words did not contribute meaningful information. The nai word is a Chinese word for nai nai(grandmother meaning), so this word appeared very much in this specific film, which is not meaningful enough for our clustering. The words “one”, “two” and “also” do not contribute much meaningful meaning for the analysis. I applied stemming to reduce related words to their base forms (such as “making”, “makes” -> “make”), and finally, whitespace was stripped to clean the text layout.

However, I didn't separate “live” and “life” into different columns, which is useful because they carry distinct meanings and are used differently across genres. For example, “live” often refers to performances or real-time events in film reviews and news articles, while “life” typically relates to themes, experiences, or philosophical discussions, especially in political or reflective writing. Merging them could blur these genre-specific patterns, reducing the accuracy of text analysis and interpretation. Keeping them separate helps preserve contextual meaning and improves the ability to distinguish between genres.

After these steps, I generated the DTM using DocumentTermMatrix(). To meet assignment requirements, I removed sparse terms using a sparsity threshold of 0.45, which, through trial and error, produced a matrix with 26 meaningful terms across 21 documents. This reduced DTM captures the most informative and frequent terms across the dataset while eliminating noise. The final matrix was saved as dtm_top25.csv and included as an appendix for reference.

One thing to note is that each genre was already carefully selected with a specific topic focus, which contributed to the high clustering accuracy in the next few questions. Because of this, I intentionally did not remove many seemingly common words such as “around” or “even.” Although I was aware that removing such words and only keeping strong keywords like “artifici” would make clustering easier and potentially achieve 100% accuracy, I chose not to do so. Instead, I wanted to challenge the model with a more realistic scenario, acknowledging that in real-world situations, clustering tasks are rarely this straightforward. Because I had tried before by just retaining all the strong terms in the DTM like artific, politic, film, this kind of words, and it shows that the accuracy is 100%, so I want to try that if I remove all the strong terms, will it still maintain high accuracy.

4.

I performed hierarchical clustering on the preprocessed corpus of 21 documents using cosine distance(distmatrix <- proxy::dist(dtm_df, method = "cosine")) and the top 26 terms remaining after sparsity reduction by using this line of code (removeSparseTerms(dtm, 0.45)) and using the inspect function and get this details for my DTM.

```
<<DocumentTermMatrix (documents: 21, terms: 26)>>
Non-/sparse entries: 361/185
Sparsity           : 34%
Maximal term length: 6
Weighting          : term frequency (tf)
```

I cut the dendrogram into 3 clusters, as the corpus contains three genres: FilmReview, News, and Political. The resulting clusters were:

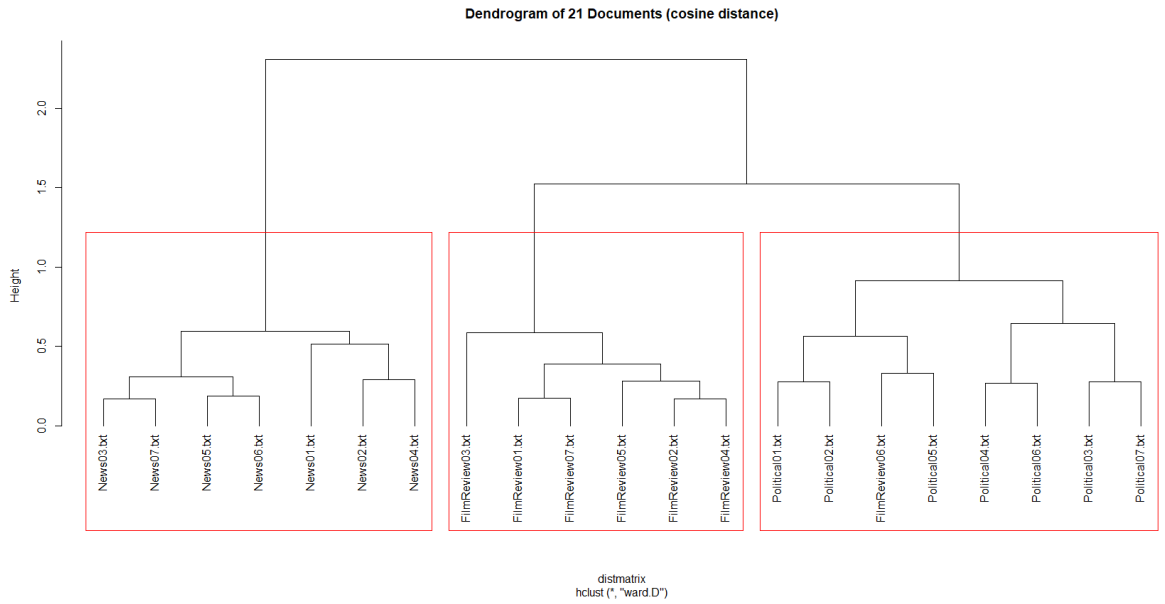
Cluster 1: 6 Film Review documents
Cluster 2: 7 Political documents + 1 FilmReview (FilmReview06.txt)
Cluster 3: All 7 News articles

	genres	cluster
FilmReview01.txt	FilmReview	1
FilmReview02.txt	FilmReview	1
FilmReview03.txt	FilmReview	1
FilmReview04.txt	FilmReview	1
FilmReview05.txt	FilmReview	1
FilmReview07.txt	FilmReview	1
FilmReview06.txt	FilmReview	2
Political01.txt	Political	2
Political02.txt	Political	2
Political03.txt	Political	2
Political04.txt	Political	2
Political05.txt	Political	2
Political06.txt	Political	2
Political07.txt	Political	2
News01.txt	News	3
News02.txt	News	3
News03.txt	News	3
News04.txt	News	3
News05.txt	News	3
News06.txt	News	3
News07.txt	News	3

This is the details on all documents belongs to which cluster

Accuracy Calculation:

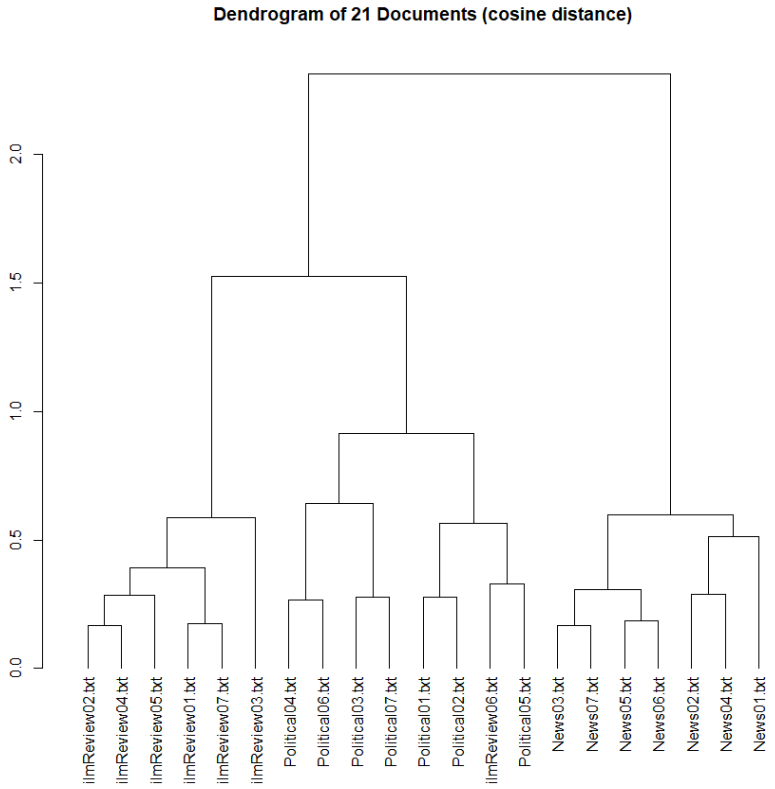
Per-genre accuracy:
Film Review: 85.71% (6/7 correct)
News: 100%
Political: 100%
Overall clustering accuracy: 95.24%



The hierarchical clustering produced strongly distinct clusters that largely align with document genres. All *News* and *Political* documents were perfectly grouped into their own clusters, indicating clear separation based on top terms like “said” in the news genre, “right” in the political genre, and “make” is also one of the top terms in the film review genre.

The only misclassification was FilmReview06.txt, which was grouped with political documents.

Even I had try to sort by cluster number, the FilmReview06.txt was still misclassification.



I use the augmented and sorted DTM, I get this table (but I have already deleted some tokens(detailed DTM) in this report for easier reading, details are in the appendix)

	end	just	use	year	genres	cluster
FilmReview01.txt	3	1	1	0	FilmReview	1
FilmReview02.txt	1	2	0	2	FilmReview	1
FilmReview03.txt	2	1	0	0	FilmReview	1
FilmReview04.txt	4	4	0	2	FilmReview	1
FilmReview05.txt	0	1	0	0	FilmReview	1
FilmReview07.txt	1	2	0	1	FilmReview	1
FilmReview06.txt	0	0	4	4	FilmReview	2
Political01.txt	0	1	0	2	Political	2
Political02.txt	0	2	1	3	Political	2
Political03.txt	2	0	3	4	Political	2
Political04.txt	1	0	0	0	Political	2
Political05.txt	0	0	1	6	Political	2
Political06.txt	1	0	0	2	Political	2
Political07.txt	2	2	1	4	Political	2

The reason why FilmReview06.txt was grouped with the Political cluster is due to the frequent use of terms such as "end," "just," "use," and "year."

In film reviews, words like "end" are common because film reviews typically follow a narrative structure that concludes with an ending and a reflection. Similarly, the word "just" appears frequently in film reviews as reviewers often explain or justify the actions of characters (e.g., "she faces love and heartbreak just like any young woman would"). I refer back to the original FilmReview06.txt, I found that the major issue is the content it, which is more on explaining the storyline but others are more on reflection for the movies. Hence, I believe that this is also a major issue to cause this misclassification and leads to no “end” and “just” words in the content of athe rticle.

On the other hand, political articles tend to focus on action plans, social issues, and policy suggestions. Words like "use" are common in political documents when discussing strategies (e.g., "Use special measures, such as legislated gender quotas and gender-balanced appointments"). Terms like "year" are also common in political documents when citing statistical studies or projecting long-term goals (e.g., "it will take 99.5 years to close the global gender gap"). As mentioned earlier, FilmReview06.txt contains frequent use of the terms "use" and "year" because the review focuses more on describing character actions in detail and referencing specific events tied to particular years. For example, the reviewer may explain what a character chose to use in certain situations or mention which year key events in the story took place. This level of specificity causes these two words to appear more commonly in the document, which in turn contributes to it being grouped with the Political cluster due to token similarity. To prevent this issue, perhaps the filmreview06 content is more about analysis, and also the reflection of the movie, not the details of the film review, which mostly describes the film itself.

This suggests that while the clustering is generally accurate, overlapping tokens across genres can influence results. Nonetheless, the overall performance and clarity of genre

separation were excellent. As the FilmReview genre has 85.71% accuracy, other genres are 100%, and the overall genres accuracy is reached to 95.24%, which is only the filmReview06.txt is misclassification.

5

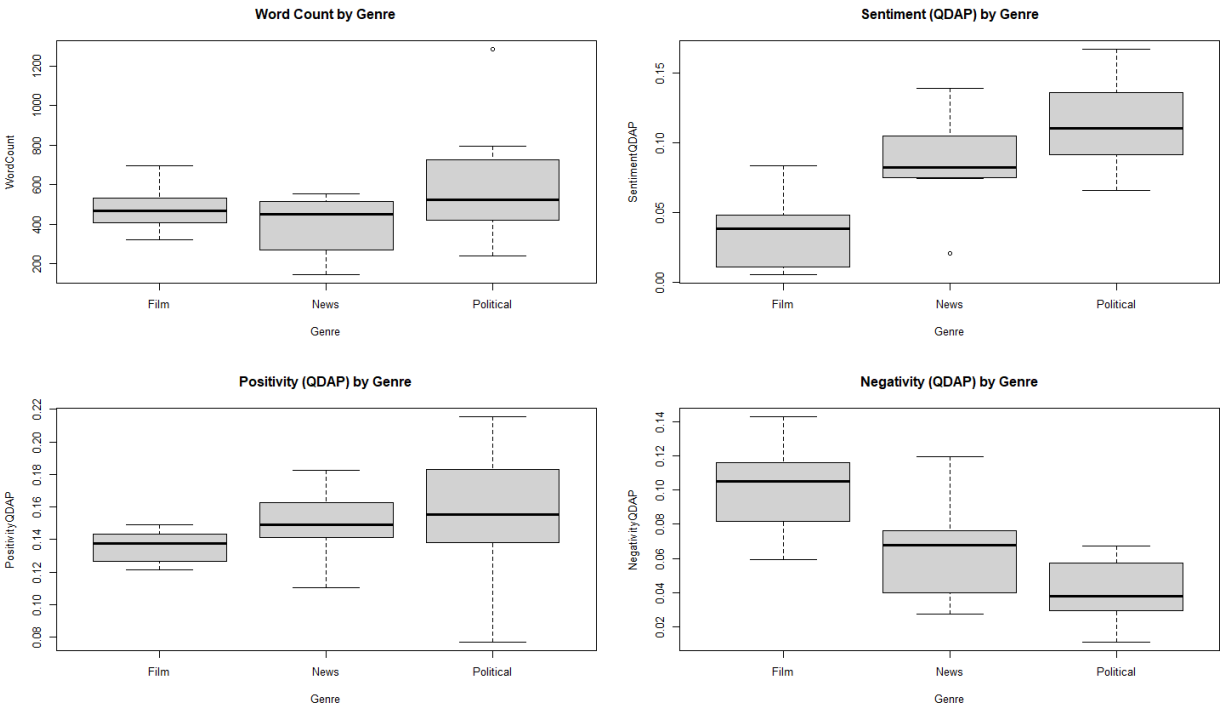
To analyse the sentiment of each document in my corpus, I used the analyzeSentiment() function from the SentimentAnalysis package in R. This function automatically applies several well-known sentiment dictionaries to the input corpus, which also includes different dictionaries (QDAP, DictionaryLM…). I use this line of code (sentiment_result <- analyzeSentiment(docs)) directly on the original 21 text files without additional preprocessing. This function automatically evaluates each document against multiple sentiment terms, including the QDAP dictionary, which I used QDAP for my analysis.

The sentiment_result gives me a raw output with sentiment scores for each document in my corpus, calculated using several sentiment dictionaries.

	WordCount	SentimentGI	NegativityGI	PositivityGI	SentimentHE	NegativityHE	PositivityHE	SentimentLM	NegativityLM	PositivityLM	RatioUncertaintyLM	SentimentQDAP	NegativityQDAP	PositivityQDAP
1	559	0.06618962	0.08407871	0.1502683	0.007155635	0.005366726	0.012522361	0.000000000	0.025044723	0.025044723	0.014311270	0.042933810	0.07871199	0.12164580
2	694	0.04466859	0.13832853	0.1829971	0.004322767	0.005763689	0.010086455	-0.031700288	0.061959654	0.030259366	0.017291066	0.005763689	0.12103746	0.12680115
3	320	0.03750000	0.14062500	0.1781250	0.009375000	0.006250000	0.015625000	-0.018750000	0.046875000	0.028125000	0.015625000	0.053125000	0.08437500	0.13750000
4	504	0.07341270	0.11507937	0.1884921	0.007936508	0.007936508	0.015873016	-0.017857143	0.047619048	0.029761905	0.021825397	0.015873016	0.11111111	0.12698413
5	371	0.12129380	0.07547170	0.1967655	0.016172507	0.010781671	0.026954178	0.013477089	0.037735849	0.051212938	0.016172507	0.083557951	0.05929919	0.14285714
6	467	0.10492505	0.12633833	0.2312634	0.010706638	0.002141328	0.012847966	-0.038543897	0.057815846	0.019271949	0.019271949	0.038543897	0.18492505	0.14346895
7	442	0.04072398	0.16289593	0.2036199	0.004524887	0.000000000	0.004524887	-0.020361991	0.040723982	0.020361991	0.004524887	0.006787330	0.14253394	0.14932127
8	145	0.15172414	0.04827586	0.2000000	0.000000000	0.013793103	0.013793103	-0.034482759	0.048275862	0.013793103	0.013793103	0.082758621	0.02758621	0.11034483
9	294	0.07482993	0.13605442	0.2108844	-0.003401361	0.010204082	0.006802721	-0.078231293	0.088435374	0.010204082	0.020408163	0.074829932	0.06802721	0.14285714
10	243	0.06584362	0.11522634	0.1810700	0.004115226	0.000000000	0.004115226	-0.082304527	0.090534979	0.008230453	0.004115226	0.020576132	0.11934156	0.13991770
11	450	0.08666667	0.09111111	0.1777778	0.011111111	0.008888889	0.020000000	-0.037777778	0.062222222	0.024444444	0.020000000	0.075555556	0.07333333	0.14888889
12	496	0.14112903	0.05846774	0.1995968	0.008064516	0.006048387	0.014112903	0.004032258	0.022177419	0.026209677	0.030241935	0.127016129	0.03629032	0.16330645
13	530	0.11886792	0.10943396	0.2283019	0.007547170	0.011320755	0.018867925	-0.033962264	0.056603774	0.022641509	0.013207547	0.083018868	0.07924528	0.16226415
14	553	0.15913201	0.07775769	0.2368897	0.016274864	0.003616637	0.019891501	-0.010849910	0.048824593	0.037974684	0.001808318	0.139240506	0.04339964	0.18264014
15	440	0.07272727	0.03636364	0.1090909	0.027272727	0.011363636	0.038636364	0.031818182	0.006818182	0.038636364	0.002272727	0.065909091	0.01136364	0.07727273
16	1285	0.15408560	0.07626459	0.2303502	0.014007782	0.003891051	0.017898833	-0.009338521	0.031128405	0.021789883	0.014007782	0.110505837	0.05058366	0.16108949
17	657	0.10350076	0.07153729	0.1750381	0.039573820	0.001522070	0.041095890	0.024353120	0.012176560	0.036529680	0.013698630	0.124809741	0.03044140	0.15525114
18	237	0.20253165	0.10126582	0.3037975	0.012658228	0.016877637	0.029535865	0.021097046	0.042194093	0.063291139	0.000000000	0.147679325	0.06751055	0.21518987
19	523	0.08030593	0.07456979	0.1548757	0.024856597	0.011472275	0.036328872	0.005736138	0.024856597	0.030592734	0.015296367	0.097514340	0.02868069	0.12619503
20	395	0.13417722	0.07594937	0.2101266	0.055696203	0.002531646	0.058227848	0.043037975	0.022784810	0.065822785	0.012658228	0.167088608	0.03797468	0.20506329
21	795	0.11069182	0.08930818	0.2000000	0.017610063	0.015094340	0.032704403	-0.028930818	0.071698113	0.042767296	0.001257862	0.085534591	0.06415094	0.14968553

However, I chose to use the QDAP dictionary because it is a general-purpose sentiment dictionary designed for quantitative and qualitative discourse analysis, rather than being limited to a specific domain like finance. Compared to DictionaryHE and DictionaryLM, which are finance-specific and more suitable for business-related or financial texts, QDAP is more appropriate for my corpus, which includes diverse genres such as film reviews, news articles, and political texts, as QDAP contains commonly used positive and negative terms suitable for general-purpose texts. Its broader applicability makes it better suited to capture sentiment across general discourse, rather than focusing on industry-specific language.

Documents were categorised into three genres: **Film**, **News**, and **Political**, based on their filenames. The main metric used for comparison was SentimentQDAP (overall polarity score), PositivityQDAP (frequency of positive words), and NegativityQDAP (frequency of negative words), representing an aggregated sentiment score from the QDAP dictionary. Additional features like WordCount are also generated to support further analysis.



From the boxplot, we can clearly see the differences in sentiment between genres

I observed the following average and variability for different genres of sentiment scores (QDAP) by having the more details table for explaining it:

Genre	Avg Sentiment	SD Sentiment	Avg Positivity	SD Positivity	Avg Negativity	SD Negativity	Avg Word Count	SD Word Count	Count
Film Reviews	0.0352	0.0282	0.136	0.0104	0.100	0.0281	480	124	7
News	0.0861	0.0388	0.150	0.0228	0.0639	0.0314	387	159	7
Political	0.1140	0.0353	0.156	0.0467	0.0415	0.0203	619	345	7

It summarizes the sentiment analysis results across three genres: Film, News, and Political. Overall, Political articles had the highest average sentiment score (0.114), followed by News (0.0861), while Film reviews had the lowest (0.0352). This indicates that political and news texts tend to use more positive or constructive language, whereas film reviews especially I had find all film which centered on death-related themes, carry a more emotionally heavy and critical tone, reflected in their higher negativity (0.100). In contrast, political texts had the

lowest negativity (0.0415) and the highest positivity (0.156), suggesting a tone of hope, advocacy, or empowerment, even when addressing serious issues like gender inequality in politics. Political articles also showed the highest variability in positivity (SD = 0.0467), likely due to a wider range of tones across different topics such as reforms, challenges, and progress. Additionally, document length appears to influence sentiment expression. Political articles had the longest average word count (619), while News was the shortest. Longer documents naturally allow for a broader range of emotional or evaluative expressions, which can lead to stronger sentiment scores and higher variability. This pattern supports the observation that genres with deeper analysis or emotional storytelling, like political text or film reviews are more expressive and sentimentally diverse, whereas news articles are often more neutral, concise, and fact-based, despite discussing impactful topics like AI replacing human jobs.

Political texts showed **higher variability** in positivity (SD = 0.0467), which may reflect the broad range of emotional tone in political content from optimistic calls for change to criticism of existing political issue.

While Film reviews had **higher variability** in negativity (SD = 0.0281), likely because films I had found is all related to death topic and reflect the life of positive meaning.

Finally, document **length (word count)** may also play a role in variability, longer texts offer more opportunity for diverse sentiment expressions, which may increase standard deviation in both positive and negative scores. For example, Political articles had the highest average word count (619 words), which may partly explain their wider range of positivity.

This shows a yes, which there are statistically **significant** differences in sentiment between genres, particularly for **overall sentiment** and **negativity,** but **not significant in positivity**. Based on the linear regression models (lm) applied:

Linear Model: SentimentQDAP ~ Genre

```
Call:
lm(formula = SentimentQDAP ~ Genre, data = sentiment_result)

Residuals:
      Min        1Q    Median        3Q       Max
-0.065566 -0.019353 -0.003384  0.017899  0.053098
Coefficients:
               Estimate Std. Error t value Pr(>|t|)
(Intercept)     0.03523    0.01300   2.709 0.014373 *
GenreNews       0.05092    0.01839   2.769 0.012655 *
GenrePolitical  0.07892    0.01839   4.292 0.000439 *
---
Signif. codes:  0 ‘*’ 0.001 ‘’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.0344 on 18 degrees of freedom
Multiple R-squared:  0.5127,     Adjusted R-squared:  0.4585
F-statistic: 9.468 on 2 and 18 DF,  p-value: 0.00155
```

Sentiment (QDAP overall polarity score):
The linear model shows that both the *News* and *Political* genres have **significantly higher average sentiment scores** than the *Film* genre.

The p-values for GenreNews (p = 0.0127) and GenrePolitical (p = 0.0004) are both below 0.05, indicating these differences are statistically significant.

The **overall model p-value** is 0.0016, with an **R^2 of 51%**, showing that genre explains a substantial portion of sentiment variability and a strong relationship.

```
Linear Model: PositivityQDAP ~ Genre

Call:

lm(formula = PositivityQDAP ~ Genre, data = sentiment_result)

Residuals:
      Min        1Q    Median        3Q       Max
-0.078405 -0.008710 -0.000427  0.012233  0.059512
Coefficients:
               Estimate Std. Error t value Pr(>|t|)
(Intercept)     0.13551    0.01156  11.724 7.34e-10 *
GenreNews       0.01452    0.01635   0.888    0.386
GenrePolitical  0.02017    0.01635   1.234    0.233
---
Signif. codes:  0 ‘*’ 0.001 ‘’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.03058 on 18 degrees of freedom
Multiple R-squared:  0.08258,    Adjusted R-squared:  -0.01936
F-statistic: 0.8101 on 2 and 18 DF,  p-value: 0.4604
```

Positivity (QDAP):
The average positivity scores are slightly higher for *News* and *Political* compared to *Film*, but the differences are **not statistically significant** (p = 0.386 for News, p = 0.233 for Political).

Political is the most positive (0.13551 + 0.02017 = 0.15568), second is **News** (0.15003).

The model p-value is 0.4604, suggesting that genre does **not significantly explain** differences in positivity because the p-value is more than 0.05.

```
Linear Model: NegativityQDAP ~ Genre

Call:
lm(formula = NegativityQDAP ~ Genre, data = sentiment_result)

Residuals:
      Min        1Q    Median        3Q       Max
-0.040986 -0.020489  0.004138  0.015356  0.055452
Coefficients:
```

```
               Estimate Std. Error t value Pr(>|t|)
(Intercept)     0.10028    0.01022   9.817 1.19e-08 *
GenreNews      -0.03640    0.01445  -2.519 0.021430 *
GenrePolitical -0.05876    0.01445  -4.067 0.000723 *
---
Signif. codes:  0 ‘*’ 0.001 ‘’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.02703 on 18 degrees of freedom
Multiple R-squared:  0.4836,     Adjusted R-squared:  0.4262
F-statistic: 8.428 on 2 and 18 DF,  p-value: 0.002612
```

Negativity (QDAP):
In contrast, *Film* reviews had significantly higher negativity than *News* and *Political* documents.
Both GenreNews (p = 0.0214) and GenrePolitical (p = 0.0007) have significantly **lower** negativity than *Film*. **Political** is the least negative (0.10028 − 0.05876 = 0.04152), second is news (0.10028 − 0.03640 = **0.06388**).
The model has an **R^2 of 48%** and an **overall p-value of 0.0026**, showing that genre is a strong predictor of negative sentiment.

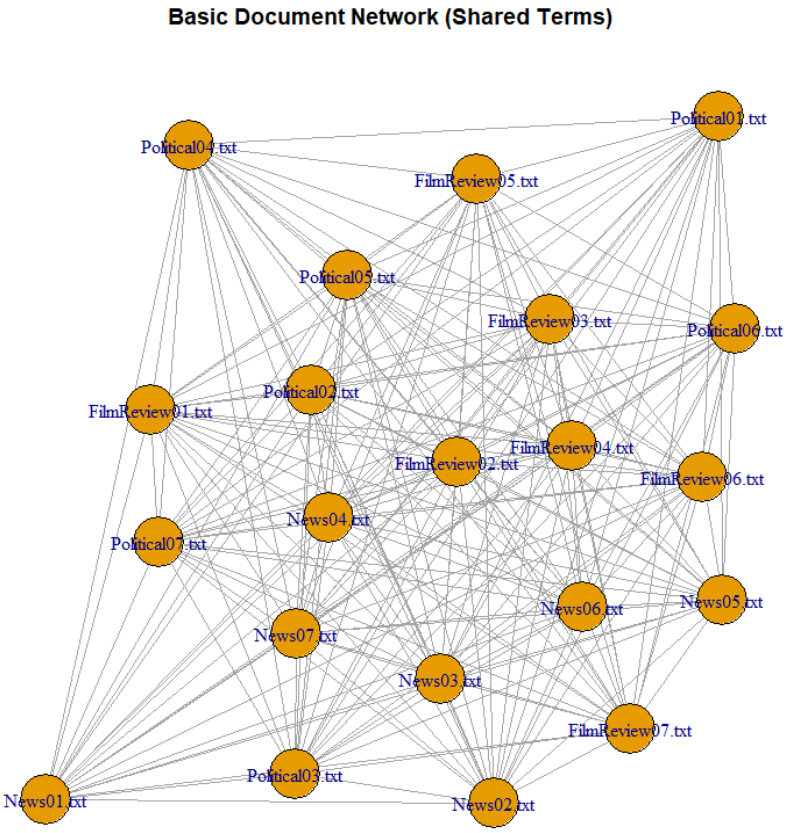
6.(calculation the connections between each document in in appendix)

The network was built from a binary document-term matrix (DTM) after text preprocessing (stopword removal, stemming…). Document similarity was calculated via matrix multiplication (`dtm_bin %*% t(dtm_bin)`), producing edge weights representing shared term counts between document pairs (e.g., FilmReview02 and FilmReview04 share 19 terms). This binary approach focuses on term co-occurrence rather than frequency, capturing broad thematic overlap. The resulting undirected graph contained 21 nodes (documents). No documents are isolated

	degree	betweenness	closeness	eig
FilmReview01.txt	20	0.0	0.006	0.822
FilmReview02.txt	20	0.0	0.005	0.993
FilmReview03.txt	20	0.0	0.006	0.879
FilmReview04.txt	20	0.0	0.005	1.000
FilmReview05.txt	20	0.0	0.008	0.658
FilmReview06.txt	20	0.0	0.005	0.921
FilmReview07.txt	20	0.0	0.006	0.824
News01.txt	20	103.3	0.011	0.352
News02.txt	20	0.0	0.006	0.769
News03.txt	20	0.0	0.006	0.816
News04.txt	20	0.0	0.005	0.960
News05.txt	20	0.0	0.005	0.928
News06.txt	20	0.0	0.006	0.924
News07.txt	20	0.0	0.006	0.960
Political01.txt	20	51.2	0.010	0.382
Political02.txt	20	0.0	0.005	0.963
Political03.txt	20	0.0	0.006	0.750
Political04.txt	20	16.0	0.008	0.455
Political05.txt	20	0.0	0.005	0.792
Political06.txt	20	0.0	0.005	0.695
Political07.txt	20	0.0	0.005	0.816

The graph shows strong interconnectivity with every document shares at least one term with all others (degree = 20 for all nodes, which n-1 edges per node since no self-loop exist), which might because the DTM is having too much of common terms like “can”, “use”, “will”, which will be appear across all genres (Film, News, Political).

The graph can be see nodes which had higher eig, then it will be more close to the center (“FilmReview04”). The nodes which had lower eig will far away from the the center of network (“News01”). The node with more high of betwenees and closeness (”News01”), will be in the farthest from the center of network because the layout_with_fr algorithm. The node with the lesser of betweeness and closeness value will be having higher of eig and closer to the center of network(“FilmReview02”, “News04”,...)



Betweenness centrality columns show that News01.txt (103.3) has the greatest, second is Political01.txt (51.2), and third is Political04.txt (16.0), because they has the weakest connections (low-weight edges), especially News01.txt, which compares to the other 21 text files. They act as hubs/bridges and facilitate between political, news, and political genres, and these nodes ensure network cohesion but operate differently from influence hubs.

So, in the graph we can see that the News01.txt, Political01.txt, Political04.txt is far from the network center, which shows that it's not strongly bonded to one genres, but it’s critical for communication across genres.

	FilmReview01.txt	FilmReview02.txt	FilmReview03.txt	FilmReview04.txt	FilmReview05.txt	FilmReview06.txt	FilmReview07.txt	News01.txt	News02.txt	News03.txt	News04.txt	News05.txt	News06.txt	News07.txt	Political01.txt	Political02.txt	Political03.txt	Political04.txt	Political05.txt	Political06.txt	Political07.txt
News01.txt	4	6	5	6	2	5	5	0	5	6	6	6	5	7	2	5	5	2	5	5	6

Closeness centrality measures how efficiently a node can interact with all other nodes in the network, reflecting its ability to quickly spread information locally. In this network, News01.txt scores highest (0.011) because its unique bridging position between different thematic clusters allows it to reach all parts of the network with minimal steps. Political01.txt follows closely (0.010), serving as another important connector between groups. While these nodes excel at facilitating communication across the entire network, other documents like FilmReview05.txt (0.008) show strong local connectivity within their own cluster but have limited influence beyond it. Most nodes score lower (≤0.006), indicating they primarily influence their immediate neighbourhood rather than the broader network.

Therefore, nodes even have high closeness centrality but still far away in the network which they are still able to reach all other documents through relatively short paths. This is because I am using layout_with_fr, so it push bridging nodes outward.

This can be see which news01.txt appears far in the network layout because it weakly connects to multiple groups without belonging strongly to any and doesn't belong tightly to any cluster, yet it has high betweenness and closeness centrality because it serves as a crucial bridge that links distant parts of the network together.

For eigencentrality, the analysis shows FilmReview04.txt (1.0) the most influential document because it maintains strong connections to other highly central nodes like FilmReview02.txt (0.993). Similarly, Political02.txt (0.963) and News07.txt (0.960) gain their high rankings through strategic ties to important nodes across different genres.

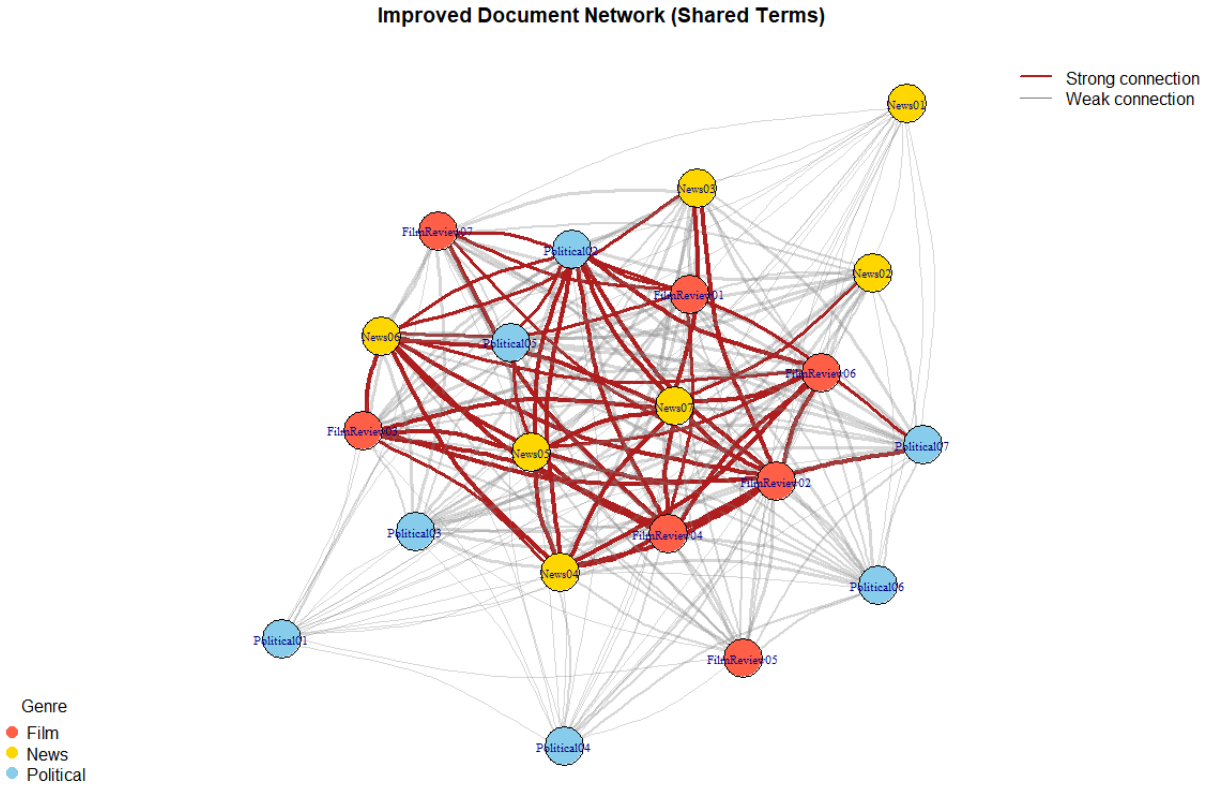
Hence, it can be see that the nodes with high eigencentrality, such as FilmReview04.txt, FilmReview02.txt is appears visually central in the network. If lower eigencentrality then the more further in the graph like Political04, Political06.

The graph average path length is 8.159 which average, it takes ~8 steps to get from one node to another. The graph diameter is 12 which show longest shortest path between any two nodes is 12. Density and clustering coefficient is 1, which shows the graph is a complete graph also, with each nodes is 20 and every doc token is connected to every doc token.

In contrast, bridge nodes like News01.txt (0.352) and Political01.txt (0.382) score low in eigencentrality despite their crucial network roles. This occurs because while they connect different clusters, their connections tend to be with less influential nodes. Their value lies in network connectivity rather than influence propagation.

The most important (central) documents in the network will fall into three key roles. If need to connect genres? News01.txt is the most important. If want the node which connected to the important nodes? FilmReview04.txt is the most important. If want strong local influencers? News01.txt is the most important again.

However, groups in the data I can identify is groups nodes by let weak connection is grey line and red colour is strong connection which can be identified two group for it with the higher and lower betweenness, revealing natural divisions in the graph.



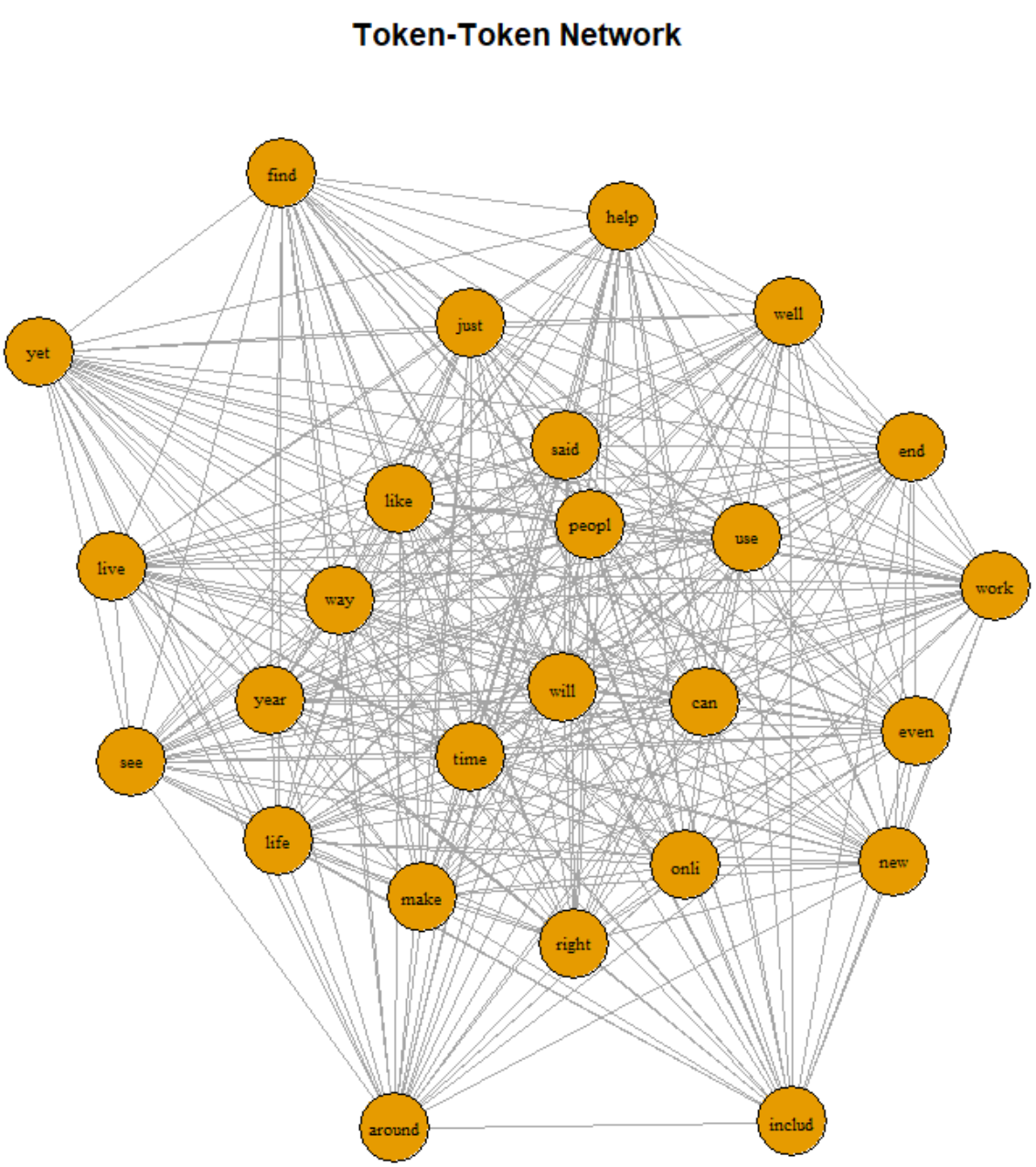
More improvement for the graphs is **for the edges** (edges with weights in the top 25% (edge weight is greater than 14) and labels them as strong connections) colored **red (firebrick) colour**, and weak ones are faded gray. So, it can be seen clearly doc has a strong connection, and the nodes which is more further away like News01.txt having the highest closeness centrality and betweenness centrality. Weak connection is with grey edges. Edge **thickness** is scaled based on how many terms documents share. If higher than it is thicker, like FilmReview04, FilmReview02.txt, which all have higher eigencentrality and a strong connection also. Then, I also added several legends to mention what is the meaning of each colour and edge.

7. (calculation the connections between each terms is in appendix)

As the same, this question just change from doc-doc to token-token in the single-mode network.

The network was also built from a binary document-term matrix (DTM) (1 = term exists in doc, 0 = not). After text preprocessing (stopword removal, stemming…). It then calculated via matrix multiplication (term_doc_matrix %*% t(term_doc_matrix)), producing edge weights representing how often two terms co-occur across different documents.

I also created an undirected graph with edge weights = co-occurrence frequency. Plotted network using layout_with_fr for the token-token network.



The resulting undirected graph contained 26 nodes (tokens). No tokens are isolated, which can be found in this table, with each token having 25 degree, and it makes each node fully connected. This is because every word appears with every other word in many documents.

	Degree	Betweenness	Closeness	Eigenvector
around	25	13.500	0.001	0.298
can	25	0.000	0.001	0.788
end	25	0.000	0.001	0.424
even	25	0.000	0.001	0.535
find	25	81.500	0.002	0.270
includ	25	110.500	0.002	0.286
just	25	0.000	0.001	0.499
like	25	0.000	0.001	0.894
live	25	0.000	0.001	0.445
make	25	0.000	0.001	0.675
new	25	2.500	0.001	0.348
right	25	0.000	0.001	0.677
said	25	0.000	0.001	1.000
see	25	0.000	0.001	0.526
time	25	0.000	0.001	0.755
use	25	0.000	0.001	0.796
way	25	0.000	0.001	0.654
well	25	0.000	0.001	0.405
help	25	0.000	0.001	0.406
life	25	0.000	0.001	0.596
onli	25	0.000	0.001	0.715
peopl	25	0.000	0.001	0.581
will	25	0.000	0.001	0.809
work	25	1.000	0.001	0.409
year	25	0.000	0.001	0.621
yet	25	42.167	0.002	0.256

The token-token co-occurrence network including density (1.0), clustering coefficient (1.0), and a uniform degree distribution of 25, confirm that every token is connected to every other token. Initially, the average path length and diameter appeared abnormally large because the co-occurrence weights were treated as similarity scores rather than distances. After correcting this by inverting the edge weights, the average path length dropped to 0.019 and the diameter to 0.042, which is consistent with the expected behavior of a complete graph.

While both the doc-doc and token-token networks are complete graphs (density = 1, clustering coefficient = 1), their average path lengths and diameters differ due to the scale and meaning of their edge weights. In the token-token graph, edge weights are based on high co-occurrence counts, resulting in very small distances after weight inversion. In contrast, document-document connections typically share fewer terms, leading to larger inverted weights (longer distances), and thus higher average path length (~8.159) and diameter (12).

The graph can be see nodes which had higher eig, then it will be more close to the center (“said”). The nodes which had lower eig will far away from the the center of network (“yet”). The node with more high of betwenees and closeness (”includ”), will be in the farthest from the center of network because the layout_with_fr algorithm. The node with the lesser of betweeness and closeness value will be having higher of eig and closer to the center of network(“will”, “time”,...)

```
Top 5 by Betweenness:
```

	Degree	Betweenness	Closeness	Eigenvector
includ	25	110.50000	0.001715266	0.2863636
find	25	81.50000	0.001655629	0.2699752
yet	25	42.16667	0.001602564	0.2562348
around	25	13.50000	0.001422475	0.2976942
new	25	2.50000	0.001373626	0.3481173

It shows that “includ” ïs the highest betweenness. These top 5 words likely bridge otherwise less-connected groups of terms, acting as connectors between genres. Even though “includ” is not the most frequent word, it’s crucial for connecting each node. It is like a semantic connector role in my corpus. If "includ" is removed from the graph, the network might become more fragmented, with fewer links between clusters.

Therefore, 5 nodes with high betweenness centrality appear in the fartest and outermost in the graph by using the layout_with_fr, so it push these bridging nodes outward.

`Top 5 by Closeness:`

	Degree	Betweenness	Closeness	Eigenvector
includ	25	110.50000	0.001715266	0.2863636
find	25	81.50000	0.001655629	0.2699752
yet	25	42.16667	0.001602564	0.2562348
around	25	13.50000	0.001422475	0.2976942
new	25	2.50000	0.001373626	0.3481173

All values are around 0.001, indicating the network is highly interconnected, which also suggests this is a very dense network where all words token are close to each other. Although all closeness values are similarly low due to the dense network, the word “includ” exhibits the highest closeness centrality, suggesting it plays a key connective role across the text. However, word terms appeared in top 5 closeness and betweness is both same, which reinforcing they are the central connector that reduces the average steps to reach other tokens. Even though they are the farthest nodes in the network.

`Top 5 by Eigenvector Centrality:`

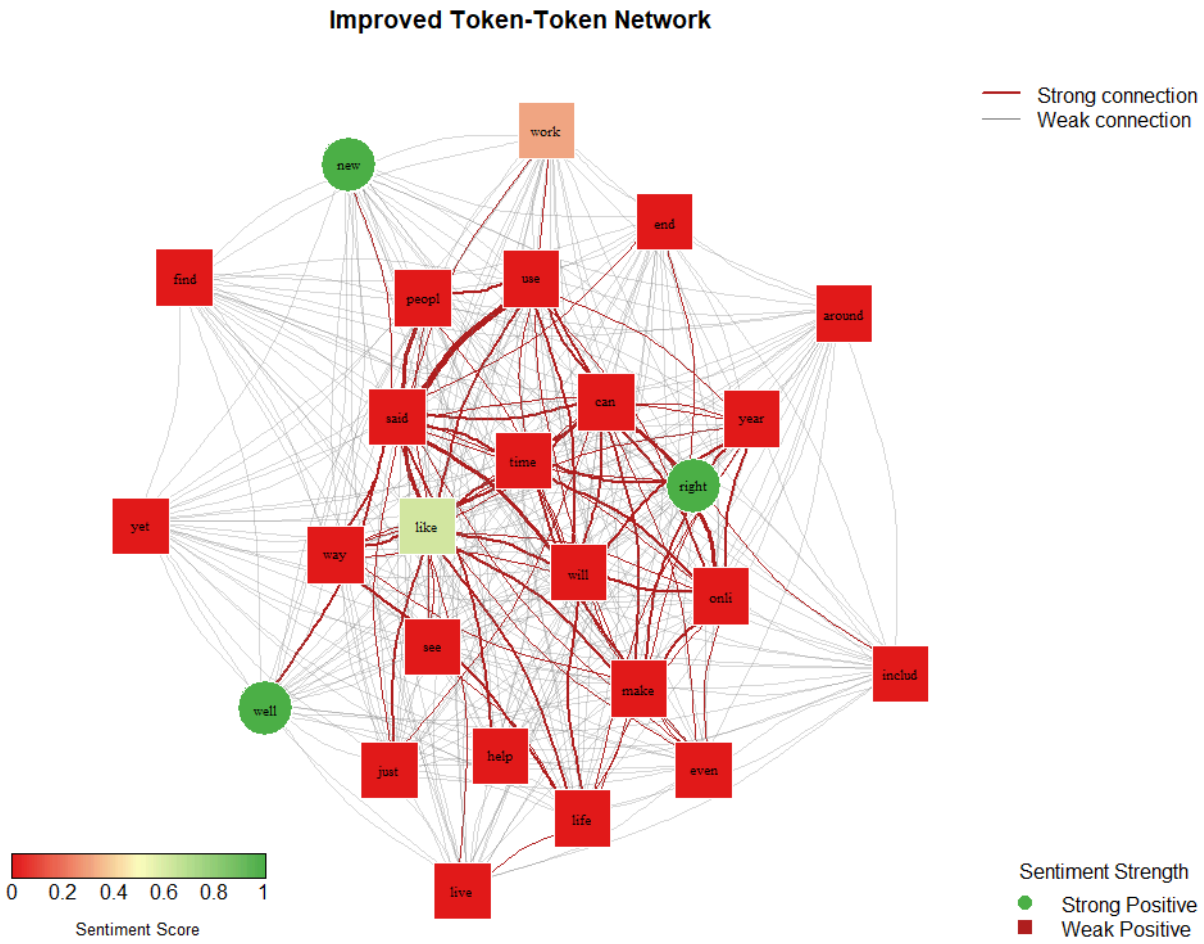
	Degree	Betweenness	Closeness	Eigenvector
said	25	0	0.0008467401	1.0000000
like	25	0	0.0006600660	0.8935200
will	25	0	0.0007401925	0.8087081
use	25	0	0.0009813543	0.7960183
can	25	0	0.0007524454	0.7879141

The greatest is “said” (1.0) in eig. It shows that these words are highly influential. Since it's frequent and well-connected in the network, it ranks 1.000 (top normalized value). “said” is the top weighted word for our corpus might because our genres(filmReview, News, Political), each text files content is more on reporting what the character in the film said, what is the people saying in the event, and who is the person encourage women participate more in political. Therefore, “said” is the top eigenvector for it and it is in the center of the network.

The top 5 word terms nodes for eigenvector also can be identified with they are the most central in the network which reinforced they has access to the network's most influential regions.

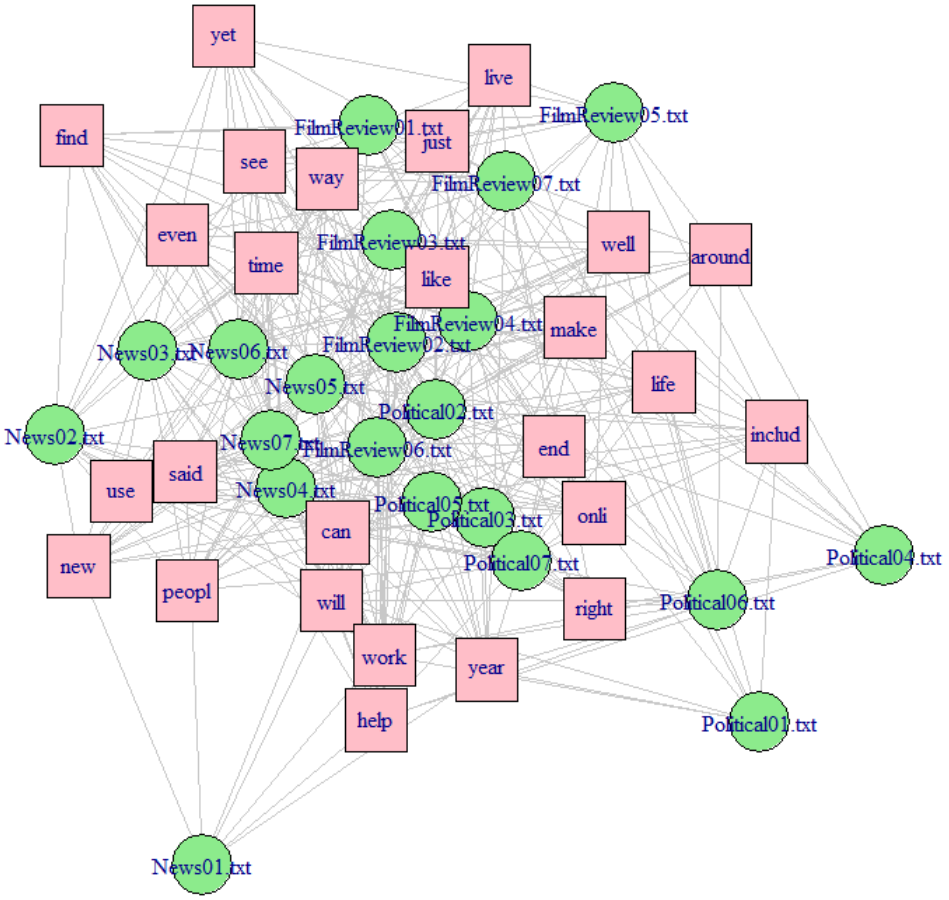
The most important (central) tokens in the network fall into three key roles based on their centrality measures. If the goal is to connect different genres and bridge diverse topics, the token "includ" is the most important. It has the highest betweenness and closeness centrality, meaning it frequently appears on the shortest paths between other tokens and is well-positioned to reach others efficiently. If we want a token that connects to many already-important nodes, then "said" is the most significant. It holds the highest eigenvector centrality, suggesting that while it may not bridge communities directly, it is deeply embedded in influential parts of the network and frequently co-occurs with other central words. Finally, if we're seeking strong local influencers, tokens that are prominent within dense neighborhoods, "includ" again plays that role due to its strategic placement across documents and genres.

However, groups in the data I can identify is groups nodes with let weak connection is grey line and red colour is strong connection which can be identified two group for it with the higher or lower betweenness, revealing natural divisions in the graph.



Thicker edges in the graph show that words co-occur more frequently. Thinner edges show words co-occur less frequently. Token with green circle show they have high sentiment score, which identified the terms is a positive word by using the QDAP library to analyse the token belongs to positive or not. It can be analyse by the sentiment score scale to know that how much of positivity for each token. For example, the more green indicate the more higher positivity for the term, but the term “like” show that it only has a little positivity for it. The red square token also show that the weak positive for them, which having 0 positivity for them. The red colour edges also same as the previous question which show that there is a strong connection, and grey show that weak connection.

8. (the transformation data is in the appendix)



In this bipartite graph, unlike the previous single-mode networks where all nodes had equal degrees, the degrees vary because documents link only to the tokens they contain, and

tokens connect to the documents where they appear. Since document lengths and token frequencies differ, node degrees are not uniform.

The **diameter** of the graph is 4, indicating that the longest shortest path between any two nodes, whether document or token, spans only four steps. This suggests that the network is relatively compact, with no nodes being excessively distant from each other. The **average path length** is approximately 2.21, meaning that, on average, it takes just over two steps to travel from one node to another. This reflects a well-connected network where information (shared tokens) can flow efficiently across documents. The **density** of the graph is 0.334, which shows that around one-third of all possible connections in the bipartite structure are present. This is a moderate density, implying that while not every token appears in every document, there is still a substantial amount of overlap in term usage across the corpus.

I generate a table (degree, betweeness, closeness and eigenvector for each nodes) to better illustrate the graph about the relationship between words and documents. Because it is contains too much information in the graph which might more difficult to explain by only words.

	Type	Degree	Betweenness	Closeness	Eigenvector
News04.txt	Document	21	91.262112	0.011363636	0.5748288
FilmReview03.txt	Document	19	67.662444	0.010989011	0.3850862
News02.txt	Document	16	55.156021	0.010416667	0.3353163
FilmReview07.txt	Document	18	51.551401	0.010638298	0.3233787
yet	Token	12	50.133932	0.010989011	0.1879686
News03.txt	Document	18	47.748346	0.010752688	0.3923445
Political06.txt	Document	15	47.156006	0.010309278	0.2279711
FilmReview01.txt	Document	18	46.801989	0.010309278	0.3934189
new	Token	14	45.429255	0.010752688	0.2602362
around	Token	13	45.289590	0.010752688	0.2181503
work	Token	14	36.254732	0.010204082	0.3064540
includ	Token	13	36.228254	0.010638298	0.2154851
help	Token	14	31.786011	0.010416667	0.3060028
FilmReview06.txt	Document	20	27.263325	0.010101010	0.5311339
Political05.txt	Document	17	26.965358	0.009708738	0.4689995
see	Token	14	26.453410	0.010638298	0.4018719
Political04.txt	Document	10	24.995585	0.009900990	0.1394986
FilmReview05.txt	Document	14	24.262472	0.009803922	0.2884798
find	Token	12	24.224917	0.010309278	0.1994463
make	Token	18	24.074356	0.010204082	0.5357147
onli	Token	16	21.589484	0.010309278	0.5807459
well	Token	12	20.155743	0.010309278	0.3053152
end	Token	14	19.226242	0.010309278	0.3224729
News07.txt	Document	21	19.191736	0.010204082	1.0000000
said	Token	16	19.088826	0.009900990	0.9715203
News06.txt	Document	20	17.109803	0.009803922	0.7736776
Political07.txt	Document	18	15.371867	0.009523810	0.5670346
Political02.txt	Document	21	15.116455	0.009615385	0.7807985
even	Token	13	14.322221	0.009803922	0.4092591
live	Token	13	14.019829	0.009615385	0.3312192
way	Token	15	13.814993	0.009900990	0.5119634
just	Token	12	13.710615	0.009708738	0.3807448
Political01.txt	Document	8	12.216112	0.008547009	0.1557936
Political03.txt	Document	16	12.098181	0.009259259	0.7939800
peopl	Token	13	10.089751	0.009803922	0.4504848
can	Token	16	10.041832	0.009708738	0.6328891
use	Token	12	9.143012	0.009009009	0.6786293
year	Token	15	8.998274	0.009433962	0.4896018
News05.txt	Document	20	8.657868	0.009433962	0.7457340
FilmReview04.txt	Document	22	7.017502	0.009523810	0.7061637
News01.txt	Document	7	6.161137	0.008547009	0.1668983
right	Token	12	5.342081	0.009615385	0.7147401
will	Token	16	4.846345	0.009259259	0.6602358
like	Token	16	4.734231	0.009174312	0.7455135
FilmReview02.txt	Document	22	3.798143	0.009433962	0.7048113
time	Token	14	1.795293	0.008928571	0.6008031
life	Token	12	1.770635	0.007812500	0.4714206

Degree is different for each node in bipartite graph which because in a bipartite doc-word network, degree counts unique connections. For document nodes, degree = number of unique tokens in the document. For token nodes, degree = number of documents the word appears in. So even if a word appears many times, it only adds one edge per document. Not same as the previous question. Each word token and document also linked it each other, and the more, which the node having lower eig value is more far away from the center in the network, the more high eig is more in the center. The graph also shows that the document

nodes (circles) are more in the center, while the word tokens (squares) appear more toward the outside. This happens because each document node connects to multiple word tokens, so it is linked by many different word nodes, bringing it toward the center of the layout. On the other hand, word tokens appear in only a few documents, meaning they have fewer connections. As a result, the layout_with_fr algorithm (like) places them closer to the documents they link to, but further away from the dense document core, especially if they are rare or unique words.

Additionally, some common words may appear in several documents and act like bridges, but rare or unique words that appear in just one or two documents will be positioned further out, since they have fewer connections to the main network.

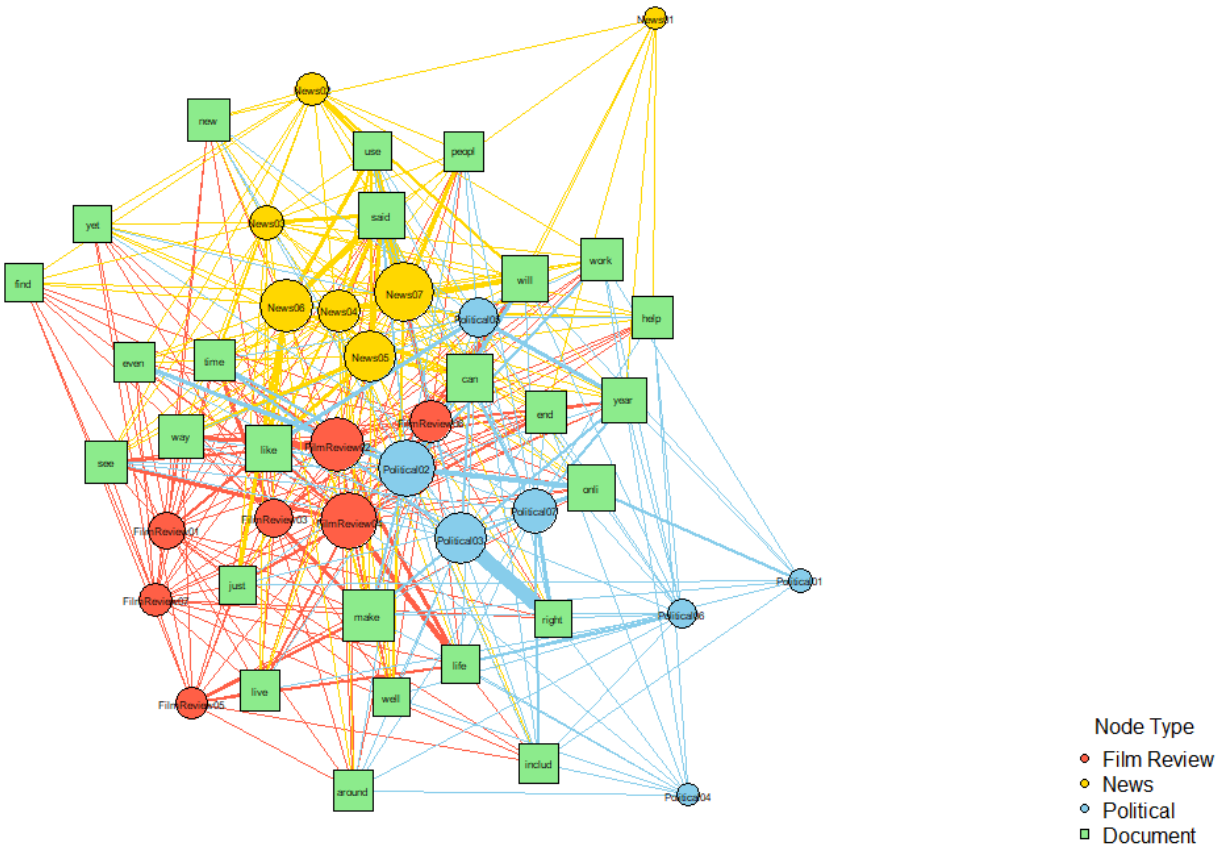
News04.txt has the highest betweenness (91.26), highest closeness (0.01136), and a relatively high eigenvector centrality (0.5748), along with a high degree of 21. This indicates that it functions as a major bridge in the network, connecting many tokens across different genres. Its high closeness centrality shows that it can quickly reach all other nodes, and even though its eigenvector score isn't the highest, it is still substantially connected to influential tokens. However, the algorithm makes News04.txt not really in the central of the network.

News07.txt, while having slightly lower betweenness, stands out with the highest eigenvector centrality (1.000). This suggests that it is not just connected to many tokens, but to highly important ones, making it an influential hub within the network. However, its lower betweenness implies that it does not act as a connector or bridge between communities. Instead, it's more like a cluster core densely tied to other strong nodes. This combination makes News07.txt in the central of the network also.

On the other hand, News01.txt has the lowest degree (7), meaning it connects to relatively few tokens. Its low betweenness and closeness values suggest that it plays a peripheral role, not serving as a connector or being particularly accessible within the network. Similarly, its lowest eigenvector centrality indicates weak influence, it's not connected to influential tokens, nor does it help bind the graph together. In short, its removal would barely disrupt network flow. As a result, you can see in the graph which News01.txt is the farthest away from the network graph.

The token "said" appears across 16 documents (degree = 16), and while its betweenness is not high (19.08), it has one of the top eigenvector scores (0.9715). This shows that it doesn't act as a bridge but is still structurally powerful by being embedded within influential parts of the network. This aligns with genre expectations "said" frequently appears in news or film review reporting styles. Hence, the "said" token is inside in the network.

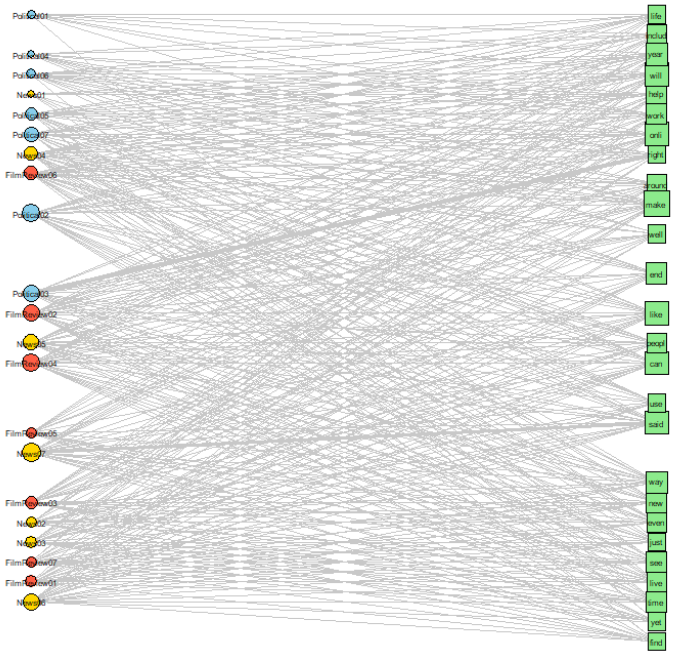
The token "yet" appears across 12 documents (degree = 12), and while its betweenness and closeness is the highest for the word token (50.133 & 0.1099), it has eigenvector scores (0.188) which the lowest in word token. Hence, we can see that the "yet" token is far away from the central of network.



In this graph, all documents are represented as circles and colored based on their genre: tomato red for Film Review, gold yellow for News, and skyblue for Political text files. Meanwhile, tokens are uniformly represented as green squares, allowing for immediate visual distinction between the two node types.

The edges connecting tokens to documents are colored according to the genre of the document they originate from. This means that if a token appears in documents from multiple genres, it will be surrounded by multi-colored edges, revealing cross-genre term usage. Additionally, edge width is scaled based on term frequency using a rescaled range from 1 to 11. Thicker edges in the graph indicate that a token is used more frequently in a particular document, while thinner edges represent lower usage. This makes us easier to identify strong vs weak document-term associations for the relationship between the document token and word token by just looking at the width of edge. Node sizes are also customized to reflect their importance. Document nodes are scaled by the total term frequency in the document, document with more words (higher total token count) appears larger like News07, FilmReview04,FilmReview02, we can clearly know they have more term appeared in their doc. While token nodes are sized by their degree (how many documents they appear in) like “make” token is the largest across all other word token, so it show a greatest size for it. Labeling has been refined for readability by removing .txt from document

labels and applying consistent font settings. Last but not least, legend is added to explain the meaning of node shapes and colors.



However, there is no any groups in the data I can clearly identify, because all the nodes is intermixed together. Even I had tried before make my bipartite graph as left is all doc and right is all words. But it still make the bipartite graph harder to trace which edges are they connected and clearly identified.

9.

In this analysis, the corpus of 21 documents grouped into three genres Film Review, News, and Political explored using both clustering and network approaches. Hierarchical clustering using cosine distance (Q4) achieved a high overall accuracy of 95.24%, clearly distinguishing genres based on dominant tokens like “said” (News), “right” (Political), and “make” (Film). However, one misclassification (*FilmReview06.txt*) occurred due to overlapping and general-purpose terms such as “year” and “use”, which blurred word terms boundaries. Clustering groups documents based on overall term similarity, which is effective when vocabulary differences are clear but less so when semantic overlap occurs.

In contrast, network analysis (Q6–Q8) provided structural and relational insights that clustering alone could not capture. In Q6 (Doc-Doc network), documents were linked based on token co-occurrence. Metrics like degree, betweenness, and eigenvector centrality revealed important nodes such as *News01.txt* and *Political01.txt*, which acted as bridges between clusters, while *FilmReview04.txt* emerged as highly influential due to its strong connectivity with other documents. Q7 (Token-Token network) emphasized semantic relationships between words. Although “said” , “like” was not the most frequent token, it had the highest eigenvector centrality, reflecting its presence in many influential documents. Meanwhile, “includ” had the highest betweenness and closeness, indicating it served as a

semantic connector across the network and often lay on shortest paths between otherwise distant concepts.

In Q8 (Bipartite Doc-Token network), documents and tokens were analyzed together in a two-mode network. Metrics identified *News04.txt* as a document with high centrality across all measures, and terms tokens like “make” having the greatest degree among the terms token, and “said” having the highest eig across all terms token were structurally prominent. Visual features such as node color, size, and edge width helped reveal genre-token associations and document richness.

To group nodes for Q6 and Q7, I use the edge betweenness clustering algorithm to detect group by use the grey edges with high betweenness, revealing natural groupings of documents with similar term patterns. The higher value of eig wil be act as a strong connection with red lines. This method was particularly helpful in identifying modular clusters that loosely align with genre, even without using pre-labeled categories.

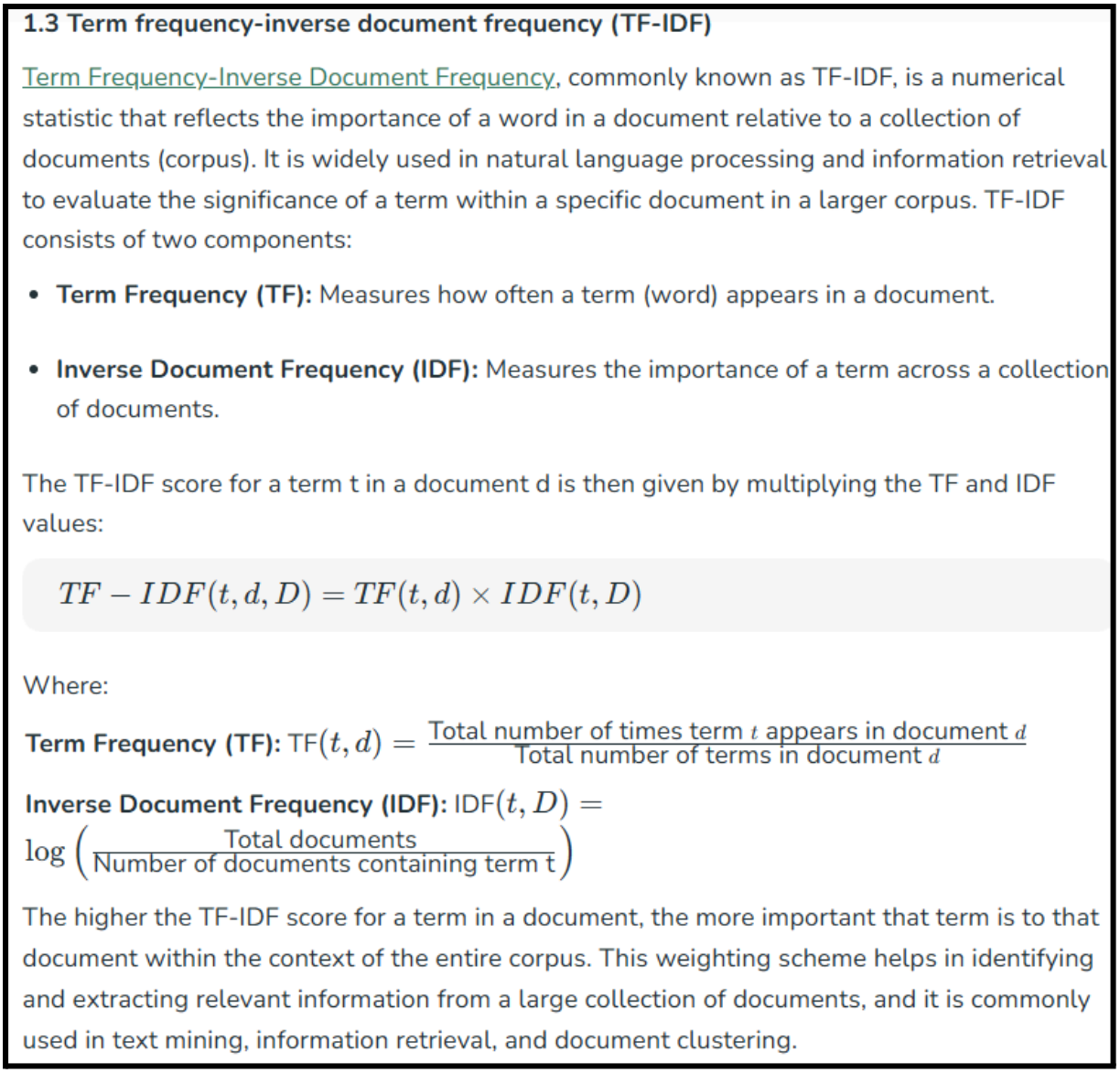
Additionally, Q5 sentiment analysis across the corpus revealed that News and Political texts leaned more neutral or formal. These often reflected informative or objective tones, consistent with their genres. In contrast, Film Reviews is used more emotional language and mostly is discussed about death, although sentiment polarity was not extreme. This alignment between structure and tone suggests that integrating sentiment measures into network attributes (edge weighting by polarity) could further enhance the interpretability of relationships across genres. The positiveQDAP is not very significant, but negativityQDAP and sentimentQDAP is significant.

When comparing both approaches, clustering and network analysis each offer unique strengths in uncovering important groups and relationships in text data. Clustering is particularly effective for genre classification based on lexical similarity, as seen in Q4 where hierarchical clustering with cosine distance achieved high accuracy in separating Film, News, and Political texts. Clustering works by grouping data points, such as documents, based on feature similarity, especially term usage. It provides clear partitions and is easy to visualize using dendrograms. However, it assumes each document belongs to one group only and does not capture complex or overlapping relationships between documents or tokens. In contrast, network analysis (applied in Q6–Q8) focuses on the relational structure of the corpus, exploring how documents or tokens are connected through shared terms. It offers valuable insights by revealing semantic bridges, structural connectors, and influential nodes that may be missed in clustering. For instance, in the token-token network, “includ” played a bridging role by having the highest betweenness and closeness, while “said” emerged as the most influential token via its top eigenvector centrality score. These findings highlight how terms function within the structure of the corpus, beyond just their frequency. Network metrics like degree, centrality, and community detection allow us to detect semantic regions and interaction-based groupings, rather than partitions based on similarity alone.

The reason network analysis is not directly compatible with clustering is that they serve different objectives. Clustering aims to partition documents based on content similarity, whereas network analysis aims to map relationships and flows within the data. While clustering is useful for identifying high-level genre structure, network analysis excels in capturing fine-grained, overlapping connections, such as which documents or tokens act as

hubs, bridges in the corpus. That said, group detection in networks can support simple grouping, but it does so by examining how nodes interact, rather than how similar their content is.

To enhance both methods especially in Q6 and Q7, replacing binary weighting with TF-IDF could significantly improve analysis. TF-IDF gives higher importance to words that are frequent in one document but rare across the corpus, which would sharpen document similarity measures and emphasize meaningful token relationships. Ultimately, while clustering efficiently organizes data into categories, network analysis reveals the hidden structure and function of terms and documents—making both methods valuable and complementary tools for text analysis.



1.3 Term frequency-inverse document frequency (TF-IDF)

Term Frequency-Inverse Document Frequency, commonly known as TF-IDF, is a numerical statistic that reflects the importance of a word in a document relative to a collection of documents (corpus). It is widely used in natural language processing and information retrieval to evaluate the significance of a term within a specific document in a larger corpus. TF-IDF consists of two components:

- **Term Frequency (TF):** Measures how often a term (word) appears in a document.
- **Inverse Document Frequency (IDF):** Measures the importance of a term across a collection of documents.

The TF-IDF score for a term t in a document d is then given by multiplying the TF and IDF values:

$$TF - IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

Where:

Term Frequency (TF): $\text{TF}(t, d) = \frac{\text{Total number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$

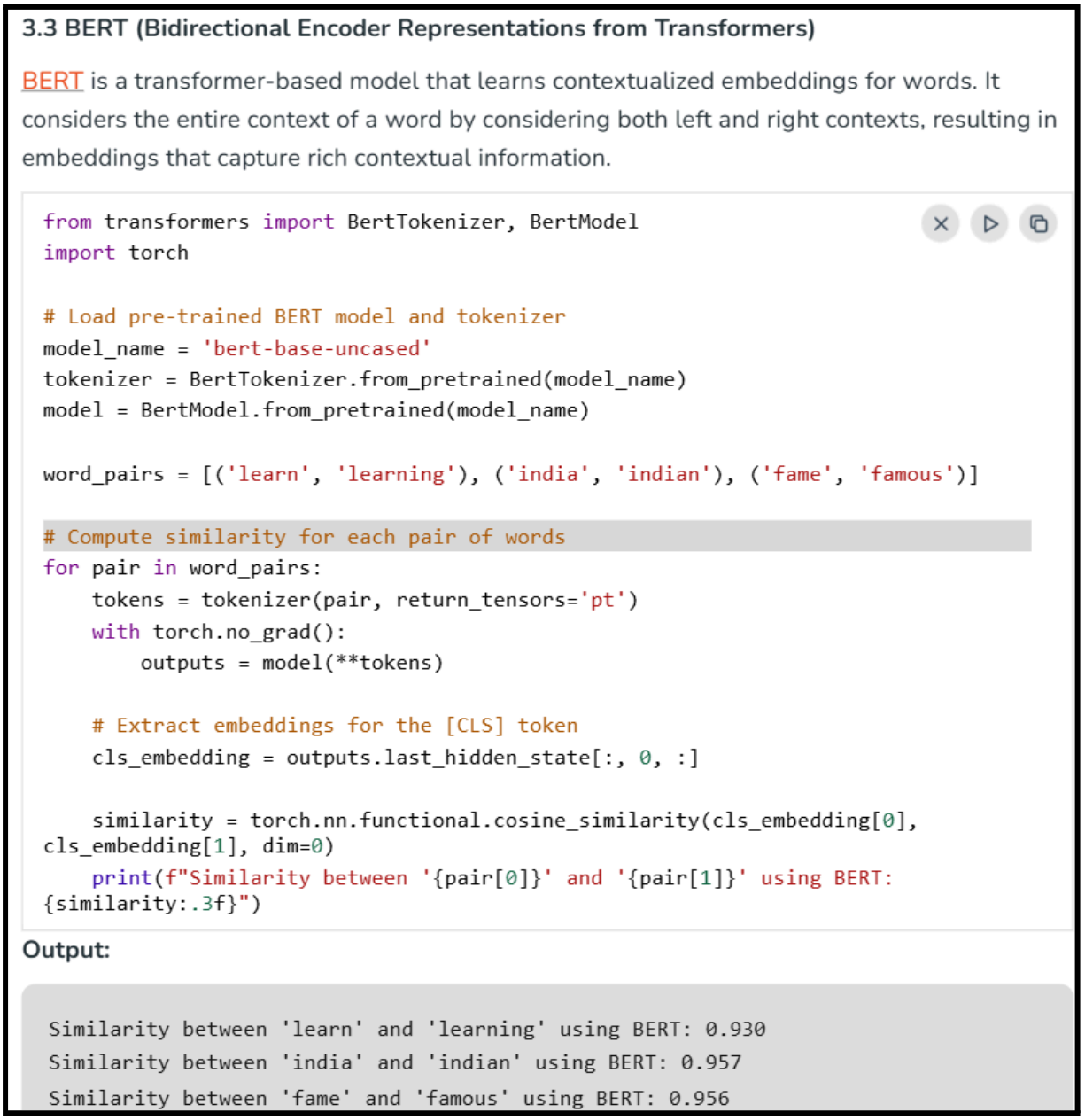
Inverse Document Frequency (IDF): $\text{IDF}(t, D) = \log\left(\frac{\text{Total documents}}{\text{Number of documents containing term t}}\right)$

The higher the TF-IDF score for a term in a document, the more important that term is to that document within the context of the entire corpus. This weighting scheme helps in identifying and extracting relevant information from a large collection of documents, and it is commonly used in text mining, information retrieval, and document clustering.

Screenshot from geeksforgeeks website:
https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp/

This would directly enhance document clustering by emphasizing unique features rather than common ones. Replacing stemming with lemmatization helps preserve correct word forms and meanings (“better” becomes “good”), which improves linguistic accuracy and

leads to more precise grouping of similar documents (Python Lemmatization Approaches with Examples, 2020). Then, incorporating bigrams or trigrams (such as “artificial intelligence” in the news genre) will captures more contextual meaning than single words, which is particularly beneficial for distinguishing topics in documents across genres and improve accuracy (TF IDF for Bigrams & Trigrams, 2019). By using term frequency filtering, straight away sort and selecting the top 25 most frequent and meaningful terms in each document. we can retain the most relevant content, improving both document accuracy and clustering quality by reducing noise from low-impact terms. It also will make the accuracy become higher and meaningful enough, instead of using some common word is often appear in all content of text such as can, just, will, which also hardly clarified and have strong evidence which genre they are correctly belong to.



3.3 BERT (Bidirectional Encoder Representations from Transformers)

BERT is a transformer-based model that learns contextualized embeddings for words. It considers the entire context of a word by considering both left and right contexts, resulting in embeddings that capture rich contextual information.

```
from transformers import BertTokenizer, BertModel
import torch

# Load pre-trained BERT model and tokenizer
model_name = 'bert-base-uncased'
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertModel.from_pretrained(model_name)

word_pairs = [('learn', 'learning'), ('india', 'indian'), ('fame', 'famous')]

# Compute similarity for each pair of words
for pair in word_pairs:
    tokens = tokenizer(pair, return_tensors='pt')
    with torch.no_grad():
        outputs = model(**tokens)

    # Extract embeddings for the [CLS] token
    cls_embedding = outputs.last_hidden_state[:, 0, :]

    similarity = torch.nn.functional.cosine_similarity(cls_embedding[0],
cls_embedding[1], dim=0)
    print(f"Similarity between '{pair[0]}' and '{pair[1]}' using BERT:
{similarity:.3f}")
```

Output:

```
Similarity between 'learn' and 'learning' using BERT: 0.930
Similarity between 'india' and 'indian' using BERT: 0.957
Similarity between 'fame' and 'famous' using BERT: 0.956
```

Screenshot from geeksforgeeks website:
https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp/

Techniques like word embeddings (BERT) could be used, because some words might not be all processed stemwords as our expectation. Words like "president" and "leader", "nation" and "citizens", or "emergency" and "crisis" are recognised as semantically similar but might not be done in the preprocessing. BERT understands their meaning in context and can remove or merge the high similarity words together, which will make my DTM perform better. Applying these techniques would improve the basic NLP and also accuracy by grouping documents with similar content / terms more effectively. This make having strong evidence the connection between each nodes is all reasonable, without using words which also will be appeared in all genres text like “can”, “will”.